

OUTUBRO | 2026

ACTA

RELATÓRIO DE JUSTIFICATIVA

2º E TECH

INTRODUÇÃO

O banco de dados do ACTA é a base **PostgreSQL** que sustenta a plataforma de melhoria contínua baseada no ciclo PDCA (*Plan, Do, Check, Act*). Ele organiza empresas, usuários e colaboradores; ciclos, problemas e causas-raiz; planos de ação, 5W2H, tarefas, metas e treinamentos; verificações, alertas e anexos.

Este relatório justifica as decisões de modelagem tomadas ao longo do projeto: por que as tabelas foram divididas dessa forma, como os relacionamentos foram definidos, quais regras o próprio banco garante e como os dados chegam até a camada analítica consumida pelo dashboard de BI.

Origem do modelo. O ACTA teve um primeiro banco de dados no primeiro ano. No segundo ano, o modelo foi redesenhado: normalizado, com restrições de integridade, auditoria e uma camada dimensional. Os dados do primeiro ano foram migrados por um script em Python (RPA), e o restante recebeu massa de dados gerada com Faker.

Escopo. O repositório *acta-database* reúne os scripts SQL numerados (001 a 009), a carga de dados em Python, o procedimento de backup e os fluxos de validação automática.

O BANCO EM NÚMEROS

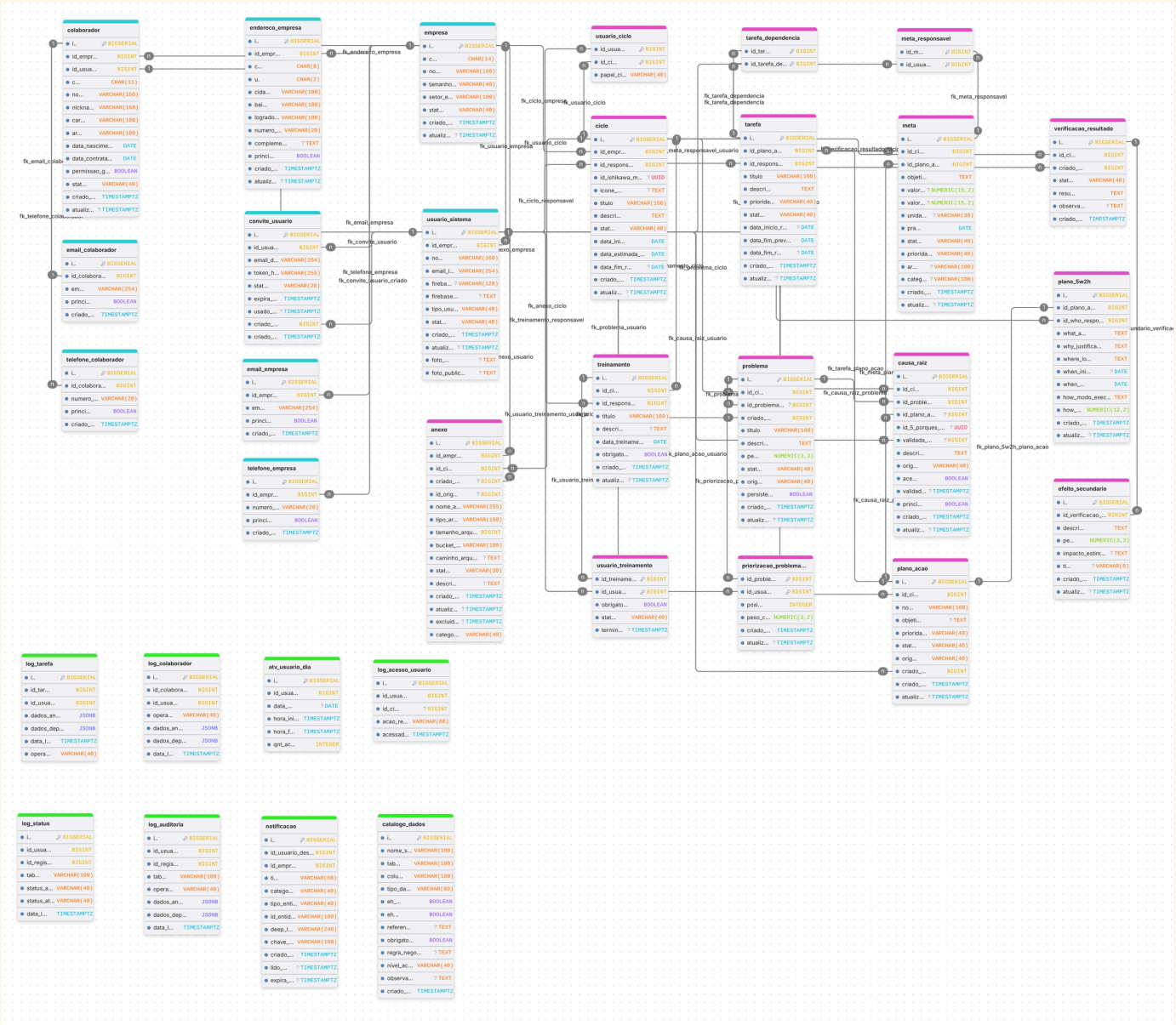
3	33	55	21	23	35
schemas	tabelas	índices	functions	triggers	views

ORGANIZAÇÃO EM TRÊS SCHEMAS

Schema	Responsabilidade	Tabelas
public (9)	Quem usa a plataforma: empresas, usuários, colaboradores, convites e contatos	`empresa` `usuario_sistema` `convite_usuario` `colaborador` `endereco_empresa` `email_empresa` `telefone_empresa` `email_colaborador` `telefone_colaborador`
pdca (16)	O núcleo do negócio: o ciclo PDCA e tudo que ele contém	`ciclo` `plano_acao` `plano_5w2h` `meta` `meta_responsavel` `tarefa` `tarefa_dependencia` `problema` `causa_raiz` `priorizacao_problema_usuario` `treinamento` `usuario_treinamento` `usuario_ciclo` `verificacao_resultado` `efeito_secundario` `anexo`
auditoria (8)	Rastreabilidade, monitoramento de uso e documentação dos dados	`log_auditoria` `log_status` `log_tarefa` `log_colaborador` `log_acesso_usuario` `atv_usuario_dia` `notificacao` `catalogo_dados`

MODELO RELACIONAL

Diagrama entidade-relacionamento do banco, com as 33 tabelas e as chaves estrangeiras. Em azul-turquesa estão as tabelas do schema `public`, em rosa as do `pdca` e em verde as de `auditoria`.



DECISÕES DE MODELAGEM

1. Normalização

Dados multivalorados foram separados em tabelas próprias. Uma empresa pode ter vários e-mails, telefones e endereços; por isso eles ficam em `email_empresa`, `telefone_empresa` e `endereco_empresa`, com um campo `principal` para indicar o contato preferencial. Isso evita colunas repetidas (telefone1, telefone2…) e atende a 1ª forma normal. Atributos dependem apenas da chave da própria tabela (2ª e 3ª formas normais): o responsável de uma tarefa é referenciado por id, e não copiado como texto.

2. Separação em schemas

Dividir em `public`, `pdca` e `auditoria` deixa claro o domínio de cada tabela, permite conceder permissões por schema (por exemplo, só leitura na auditoria) e evita colisão de nomes.

3. Usuário separado de colaborador

`usuario_sistema` guarda o que é necessário para login e perfil (e-mail, tipo ADMIN, GESTOR ou COLABORADOR, Firebase UID). `colaborador` guarda dados pessoais e funcionais (CPF, cargo, área). Assim a autenticação é desacoplada dos dados sensíveis de RH, e um usuário administrativo não precisa ter CPF.

4. Escolha dos tipos de dados

`BIGSERIAL` nas chaves; `CHAR(14)` e `CHAR(11)` para CNPJ e CPF, que têm tamanho fixo e são guardados só com números; `TIMESTAMPTZ` em datas de auditoria, para manter o fuso; `NUMERIC` em valores e pesos, para evitar erro de arredondamento; `JSONB` nos logs, que guardam o registro inteiro antes e depois de uma alteração.

5. Modelo híbrido com MongoDB

Diagramas Ishikawa e análises dos 5 Porquês têm estrutura variável e aninhada. Em vez de forçá-los em colunas, ficam em documentos no MongoDB e o banco relacional guarda só a referência (`id_ishikawa_mongo` em `ciclo` e `id_5_porques_mongo` em `causa_raiz`, ambos UUID). O PostgreSQL continua sendo a fonte dos dados que precisam de integridade e análise.

6. Anexos com origem genérica

`anexo` usa `categoria` e `id_origem` para apontar para planos, problemas, metas, causas-raiz e outros. Evita criar uma tabela de anexos por entidade. O custo é não haver FK para `id_origem`, compensado por um `CHECK` que exige a origem sempre que a categoria não for “OUTRO” e por um limite de tipos de arquivo aceitos.

INTEGRIDADE E REGRAS

A integridade fica no banco, e não só na aplicação. Assim web, mobile, scripts de carga e qualquer outro cliente seguem as mesmas regras.

RESTRIÇÕES DECLARATIVAS

Tipo	Exemplo no modelo	Por quê
NOT NULL	Datas previstas de ciclo e tarefa, título, status	Campos essenciais nunca ficam vazios
UNIQUE	cnpj, cpf, email_login, firebase_uid, token_hash	Evita duplicidade de identidade
CHECK de formato	CNPJ com 14 dígitos, CPF com 11, e-mail por regex, CEP com 8	Dado já entra padronizado, o que facilita junções e análises
CHECK de domínio	status do ciclo, prioridade da tarefa, papel no ciclo	Só estados previstos pelo negócio
CHECK de coerência	data_fim ≥ data_inicio; colaborador com 18+ anos; peso entre 0 e 1; custo ≥ 0	Impede registros contraditórios
CHECK condicional	Usuário ATIVO exige firebase_uid; convite USADO exige usado_em	Regra depende do estado do registro

AÇÕES REFERENCIAIS (ON DELETE)

- **CASCADE** nas partes que não existem sem o todo: apagar um ciclo remove seus planos, metas e treinamentos; apagar um plano remove tarefas e o 5W2H.
- **RESTRICT** onde apagar causaria perda de histórico: quem criou um convite e os anexos de um ciclo não podem ser removidos silenciosamente.
- **SET NULL** em `anexo.criado_por`: o arquivo permanece mesmo se o autor sair.

REGRAS DE NEGÓCIO NO BANCO

Entre as regras implementadas no banco, destacamos aqui apenas as mais importantes. A lista completa está em *003_functions_triggers_procedures.sql*.

Tipo	Nome	O que garante
Function	`fn_avanco_ciclo`	Calcula o percentual de avanço de um ciclo com base nas tarefas concluídas ou canceladas
Function	`fn_pode_encerrar_ciclo`	Só permite encerrar um ciclo quando não há planos de ação nem tarefas pendentes
Trigger	`tg_status_plano_*`	Mantém o status do plano de ação coerente com o status das suas tarefas
Trigger	`tg_validar_dependencia_tarefa`	Impede que uma tarefa dependa de si mesma ou de uma tarefa que já depende dela
Procedure	`pr_gerar_alertas_atraso`	Marca tarefas vencidas como atrasadas e gera a notificação ao responsável, sem duplicar
Procedure	`pr_encerrar_ciclo`	Encerra o ciclo e registra a data real de fim, após validar com `fn_pode_encerrar_ciclo`

AUDITORIA E GOVERNANÇA

AUDITORIA

Triggers registram automaticamente cada `INSERT`, `UPDATE` e `DELETE` das tabelas sensíveis, guardando o estado anterior e o posterior em **JSONB**. Mudanças de status ganham um log próprio (`log_status`), o que permite medir quanto tempo um item ficou em cada estado.

Para saber **quem** fez a alteração, o backend informa o usuário na mesma transação com `SET LOCAL app.current_user_id`. Se a informação não chegar, o banco recusa a operação. Isso impede alterações sem responsável identificado.

As tabelas de log **não têm chave estrangeira** de propósito. Um log deve sobreviver à exclusão do registro que ele descreve; com FK, apagar um usuário ou tarefa apagaria também a prova do que aconteceu.

MONITORAMENTO DE USO

`atv_usuario_dia` e `log_acesso_usuario` alimentam views de usuários ativos por dia (DAU) por empresa e usuário, acessos a ciclos, convites e anexos. Essa camada permite acompanhar como a plataforma está sendo usada.

CATÁLOGO E COMENTÁRIOS

`auditoria.catalogo_dados` descreve cada coluna do banco: tipo, se é PK ou FK, referência, obrigatoriedade, regra de negócio e **nível de acesso**. Essa classificação busca seguir a LGPD, indicando quais dados exigem mais cuidado. Os mesmos textos estão como `COMMENT` nativo do PostgreSQL, visíveis em qualquer ferramenta de administração.

BACKUP E RECUPERAÇÃO

O procedimento está documentado no repositório (*Procedimentos_backup.md*): backup diário e obrigatório antes de qualquer deploy, com `pg_dump -Fc`; verificação do arquivo com `pg_restore --list`; restauração primeiro em um banco de teste e testes de conferência após a recuperação. O script `001_script.sql` recria os schemas do zero, por isso a documentação alerta para usá-lo apenas em banco descartável.

VALIDAÇÃO AUTOMÁTICA

Workflows do GitHub Actions sobem um PostgreSQL temporário a cada *pull request* e executam os scripts em ordem, impedindo que uma alteração quebre a estrutura do banco antes de chegar à produção.

DESEMPENHO E ÍNDICES

O PostgreSQL cria índice automático para chaves primárias e restrições `UNIQUE`, mas **não** para chaves estrangeiras. Como o ACTA consulta quase sempre por empresa, ciclo, responsável e status, foram criados **55 índices** sobre FKs e colunas de filtro frequente.

Cada índice foi medido com `EXPLAIN ANALYZE` antes e depois, e o resultado está registrado em *002_indices.sql*. Os casos de maior ganho:

Consulta	Antes (ms)	Depois (ms)
Causa-raiz por ciclo	558,289	0,049
Meta por área	329,462	0,027
Verificação por status	118,682	0,037
Colaborador por cargo	116,626	0,029
Endereço por empresa	98,828	0,090
Colaborador por empresa	96,731	0,162
Problema por ciclo	93,369	0,032
Meta por status	89,230	0,078
Ciclo por empresa	39,085	0,604

ESCOLHAS DE INDEXAÇÃO

- **Chaves estrangeiras e colunas de filtro.** Os índices foram criados onde o banco mais consulta ou junta tabelas: relacionamentos entre entidades e colunas usadas em filtros, como status, prioridade e prazo.
- **Tabelas N:N nos dois sentidos.** A chave primária composta cobre a busca em um sentido; índices no sentido contrário evitam varreduras quando a consulta parte do outro lado da relação.
- **Índices parciais e compostos.** Quando a consulta filtra um subconjunto frequente ou já exige uma ordenação, o índice é criado para esse caso, ficando menor e evitando o passo de ordenação.
- **Medir antes de criar.** Todo índice passou por `EXPLAIN ANALYZE`, evitando índices que só aumentariam o custo de escrita sem ganho de leitura.

Observação: as medições foram feitas na massa de teste atual (500+ registros). Os tempos absolutos mudam com o volume, mas o princípio vale: sem índice o custo cresce com a tabela; com índice, quase não cresce. Alguns "antes" altos podem incluir cache frio.

CAMADA ANALÍTICA

O modelo relacional foi pensado para gravar com consistência; o BI precisa ler e agregar. Para não misturar as duas necessidades, o modelo expõe uma **camada dimensional em views**, que o pipeline do Databricks consome na ingestão (camada Bronze).

MODELO DIMENSIONAL

Camada	Views	Papel
Dimensões	vw_dim_usuario, vw_dim_empresa, vw_dim_ciclo, vw_dim_plano_acao, vw_dim_data	Descrevem “quem, onde e quando”, com só as colunas úteis para filtros
Fatos	vw_fato_tarefas, vw_fato_planos_acao, vw_fato_treinamentos, vw_fato_alertas, vw_fato_logs_auditoria	Registram os eventos mensuráveis, ligados às dimensões por id
Análises	vw_resumo_ciclo, vw_tarefas_prazo, vw_desempenho_responsavel, vw_acompanhamento_metas, vw_ranking_problemas	Indicadores prontos: prazos, metas e desempenho (CTEs)
Hierarquias	vw_arvore_problemas, vw_dependencias_tarefas, vw_tarefas_dependentes	Percorrem problemas e dependências em qualquer profundidade (CTEs recursivas)
Séries e rankings	vw_aderencia_prazo_ciclo, vw_engajamento_treinamento, vw_evolucao_atividade_usuario, vw_historico_verificacao_ciclo	Rankings, acumulados e comparações com o período anterior (Window Functions)
Monitoramento	vw_dau_por_empresa, vw_dau_detalhado_usuario, vw_painel_auditoria, vw_monitoramento_convites e anexos	Uso da plataforma e trilha de auditoria

POR QUE VIEWS E NÃO TABELAS SEPARADAS?

- **Sem duplicar dados:** a view reflete sempre o estado atual do banco.
- **Dados pessoais protegidos:** as dimensões expõem só o necessário (a dimensão de usuário não inclui CPF nem Firebase UID).
- **Complexidade escondida:** junções e cálculos ficam no banco, e a pipeline do Databricks apenas copia e trata os dados.
- **Nomes de colunas voltados a BI:** chaves como `id_dim_empresa` deixam explícito como ligar fatos e dimensões.

TÉCNICAS SQL USADAS

CTEs deixam consultas longas legíveis em etapas. **CTEs recursivas** resolvem a árvore de problemas e a cadeia de dependências entre tarefas, o que um `JOIN` simples não faria com profundidade desconhecida. **Window Functions** calculam ranking, acumulado e variação sem perder o detalhe de cada linha.

ENGENHARIA DE DADOS

CARGA E MIGRAÇÃO DOS DADOS

Dois scripts em Python alimentam o banco. O notebook `dataload.ipynb` gera a massa de dados de teste com Faker (pt-BR) e é executado com **Papermill**, que permite rodar o notebook de forma parametrizada e automática, inclusive nos fluxos de validação do GitHub. O script `rpa.py` migra os dados do banco do primeiro ano para o modelo novo, usando Pandas e SQLAlchemy.

DECISÕES DA CARGA E DA MIGRAÇÃO

- **Ordem das dependências.** Empresas antes de usuários, usuários antes de ciclos, ciclos antes de planos, e assim por diante. Respeitar a ordem evita violação de chave estrangeira.
- **Remapeamento de IDs.** Os IDs do banco antigo não são reaproveitados: o RPA pega novos números da sequência e guarda um mapa "id antigo → id novo" para ajustar as chaves estrangeiras das tabelas seguintes.
- **Saneamento.** CPF, CNPJ, telefone e CEP são reduzidos a dígitos e os e-mails são normalizados, para passarem nas restrições do novo modelo.
- **Transação e auditoria.** Cada tabela é carregada em transação própria, com `app.current_user_id` definido, de modo que a migração também fica registrada na auditoria. Falhas encerram o processo com erro, sem carga parcial silenciosa.
- **Sequências.** Ao final, as sequências do PostgreSQL são atualizadas, para que novos registros não colidam com os migrados.
- **Dados verossímeis.** A massa de teste usa pesos e probabilidades (ex.: mais ciclos nos estados intermediários que nos finais) para se aproximar de uma operação real, o que torna os gráficos do dashboard mais representativos.

CONCLUSÃO

RESUMO DAS JUSTIFICATIVAS

- **Consistência primeiro.** Restrições, chaves e triggers garantem no banco o que o negócio exige, independentemente do cliente que escreve.
- **Normalização com pragmatismo.** Tabelas separadas onde há repetição ou N:N; tipos e status simples onde uma tabela de domínio só traria complexidade.
- **Rastreabilidade.** Auditoria com JSONB, logs independentes e catálogo de dados com nível de acesso.
- **Desempenho medido.** Índices criados a partir de consultas reais e validados com `EXPLAIN ANALYZE`.
- **Pronto para BI.** Dimensões e fatos em views alimentam o pipeline do Databricks, que entrega as tabelas Gold ao dashboard.

LIMITAÇÕES E PRÓXIMOS PASSOS

- Os dados são **ilustrativos**, gerados para validar o modelo, e o volume (500+ registros) é pequeno para conclusões de desempenho em escala.
- `anexo.id_origem` não tem FK por ser polimórfico. Uma alternativa futura seria tabelas associativas por tipo de origem.
- As referências ao MongoDB (Ishikawa e 5 Porquês) não são validadas pelo PostgreSQL; a consistência depende da aplicação.
- As views são calculadas a cada consulta. Com mais dados, vale avaliar *materialized views* ou tabelas incrementais.
- O `001_script.sql` recria o banco do zero; para produção, adotar migrações incrementais versionadas.

Mesmo com essas limitações, o modelo atende ao propósito do ACTA: sustentar o ciclo PDCA com dados íntegros, auditáveis e prontos para análise.

ONDE ENCONTRAR NO REPOSITÓRIO

Repositório completo: https://github.com/AppActa/acta-database.git (pasta **sql/**: banco; **python/**: carga e migração; **doc/**: backup, relatório e diagrama; **.github/workflows/**: validação)

O que	Link no GitHub
Criação dos schemas, tabelas, chaves e restrições	https://github.com/AppActa/acta-database/blob/main/sql/001_script.sql
Índices com `EXPLAIN ANALYZE` (antes e depois)	https://github.com/AppActa/acta-database/blob/main/sql/002_indices.sql
Functions, triggers e procedures (regras de negócio e auditoria)	https://github.com/AppActa/acta-database/blob/main/sql/003_functions_triggers_procedures.sql
Views com CTEs	https://github.com/AppActa/acta-database/blob/main/sql/004_ctes.sql
Views com CTEs recursivas	https://github.com/AppActa/acta-database/blob/main/sql/005_recursive_ctes.sql
Views com Window Functions	https://github.com/AppActa/acta-database/blob/main/sql/006_window_function.sql
Comentários nativos nas tabelas e colunas	https://github.com/AppActa/acta-database/blob/main/sql/007_comments.sql
Catálogo de dados (nível de acesso, regras)	https://github.com/AppActa/acta-database/blob/main/sql/008_catalogo.sql
Dimensões, fatos e views de monitoramento	https://github.com/AppActa/acta-database/blob/main/sql/009_views.sql
Carga de dados de teste (Faker)	https://github.com/AppActa/acta-database/blob/main/python/dataload.ipynb
Migração do banco do primeiro ano (RPA)	https://github.com/AppActa/acta-database/blob/main/python/rpa.py
Dependências Python	https://github.com/AppActa/acta-database/blob/main/python/requirements.txt
Backup e restauração	https://github.com/AppActa/acta-database/blob/main/doc/Procedimentos_backup.md
Este relatório	https://github.com/AppActa/acta-database/blob/main/doc/Relat%C3%B3rio%20de%20Justificativa%20-%20ACTA.pdf
Diagrama do banco (modelagem de dados)	https://github.com/AppActa/acta-database/blob/main/doc/Banco_SQL_ACTA.svg
Validação automática de scripts (desenvolvimento)	https://github.com/AppActa/acta-database/blob/main/.github/workflows/deploy-dev.yml
Validação automática de scripts (produção)	https://github.com/AppActa/acta-database/blob/main/.github/workflows/deploy-producao.yml
Instruções de execução e visão geral	https://github.com/AppActa/acta-database/blob/main/README.md
Modelo de variáveis de ambiente	https://github.com/AppActa/acta-database/blob/main/.env.example